INFORME TÉCNICO FINAL

AI Dev Tools Zoomcamp

Agente de revisión de código asistido por IA y servidor MCP

Variante C: desarrollo sin Claude API

Proveedor de modelo: Groq API

Modelo configurado: openai/gpt-oss-20b

Servidor MCP: Python + FastMCP

Estudiante: Víctor Manuel Cañas Salazar

Fecha de entrega: 16 de octubre de 2026

1. Resumen ejecutivo

El proyecto construye una base para revisar código con ayuda de un modelo de lenguaje y capacidades expuestas por un servidor del Model Context Protocol (MCP). Debido a que no se dispuso de acceso a Claude API, la universidad autorizó la Variante C. Se utilizó Groq y se configuró el modelo openai/gpt-oss-20b.

El repositorio contiene un servidor MCP en Python con cinco herramientas, dos recursos y dos prompts; un agente que realiza una llamada al modelo y organiza una respuesta; un conjunto de 15 casos de evaluación; resultados JSON de varias ejecuciones; pruebas automatizadas de servidor y validación de rutas; capturas del MCP Inspector; y un flujo de GitHub Actions.

Las cinco pruebas locales pasaron. Según lo reportado por el estudiante, después de fijar la dependencia de MCP a una versión compatible, las pruebas de GitHub Actions también quedaron correctas. La primera ejecución remota falló porque el archivo de dependencias permitía instalar MCP 2.x, incompatible con la importación FastMCP usada por el código. Se corrigió limitando la dependencia a MCP 1.x.

La evaluación posterior a la aclaración de la especificación del caso LOG-04 registró 11 verdaderos positivos, 0 falsos positivos y 0 falsos negativos a nivel de caso, con precisión, recall y F1 de 1.00 y tasa de falsos positivos en controles limpios de 0.00. Esos números no deben interpretarse como una medida definitiva de exactitud de cada hallazgo: el evaluador clasifica la detección a nivel de caso/categoría y no valida exhaustivamente que el hallazgo coincida con el defecto exacto esperado.

La solución es una implementación funcional parcial y documentada. No se afirma que exista una integración completa agente-MCP, un sandbox fuerte para ejecutar pruebas, un sistema de comentarios de revisión en pull requests ni un análisis de costes facturados, porque no hay evidencia suficiente para certificar esas funciones.

Resumen de estado

Área	Estado respaldado
Servidor MCP	Cinco herramientas, dos recursos y dos prompts definidos; pruebas y capturas disponibles.
Modelo alternativo	Conexión Groq y prueba mínima de llamada a herramienta realizadas.
Evaluación	15 casos y varios JSON de ejecución; métrica principal a nivel de caso.

Seguridad	Pruebas de rutas y filtros básicos; no equivale a aislamiento fuerte.
GitHub Actions	Workflow de pytest publicado; la ejecución más reciente pasó.
Informe	Este documento describe arquitectura, resultados, límites, riesgos y preguntas conceptuales.

2. Contexto, problema y objetivos

La revisión manual de código puede pasar por alto errores de lógica, validaciones incompletas, riesgos de seguridad y problemas de concurrencia o rendimiento. Un modelo de lenguaje puede ayudar a priorizar posibles defectos, pero sus salidas deben verificarse con herramientas, pruebas y criterios de evaluación reproducibles.

Objetivo general

Desarrollar y evaluar una base de revisión de código asistida por IA que use un modelo alternativo autorizado, exponga capacidades de inspección mediante MCP y permita registrar resultados de un conjunto de pruebas conocido.

Objetivos específicos

- Crear un servidor MCP en Python que exponga operaciones de lectura e inspección del proyecto.
- Publicar recursos y prompts reutilizables para revisiones generales y de seguridad.
- Conectar un cliente mínimo con Groq y el modelo configurado.
- Evaluar 15 casos agrupados por tipos de defecto y controles limpios.
- Implementar pruebas de seguridad de rutas y registrar evidencias.
- Automatizar pruebas en GitHub Actions.
- Documentar resultados, decisiones, limitaciones y conceptos relevantes.

Variante autorizada

La ruta alternativa fue necesaria por la falta de acceso a Claude API. La alternativa seleccionada fue Groq con el modelo openai/gpt-oss-20b. La selección se comunicó a la universidad. El cambio de proveedor no elimina los objetivos de MCP, evaluación, seguridad ni documentación.

3. Arquitectura de la solución

El repositorio separa el servidor MCP, el agente, el evaluador, los casos, las pruebas y las evidencias. Esta separación facilita probar los componentes por separado, aunque no debe confundirse con una integración completa de extremo a extremo.

Componente
Archivo o carpeta
Responsabilidad

Servidor MCP
mcp_server/server.py
Expone herramientas, recursos y prompts; aplica controles de rutas y límites de lectura.

Agente
agent/agent.py
Realiza una solicitud al modelo Groq y organiza el resultado de revisión.

Casos
eval/casos/casos.json
Define 15 entradas de evaluación con defectos y controles.

Evaluador
eval/run_eval.py
Ejecuta los casos y calcula métricas de detección a nivel de caso.

Resultados
eval/resultados/*.json
Conserva resultados de ejecuciones previas y posteriores.

Pruebas
tests/
Verifica funciones del servidor y validación de rutas.

Evidencias
evidencias/
Capturas de MCP Inspector y registros de pruebas.

CI
.github/workflows/ci.yml
Instala dependencias y ejecuta pytest en un runner de GitHub.

Flujo conceptual

1. Se proporciona código o un caso de evaluación. 2. El agente envía una solicitud al modelo. 3. La respuesta se transforma en una estructura de hallazgos. 4. El evaluador

determina si el caso se detectó según su regla actual. 5. El resultado y las métricas se guardan en JSON.

Limitación importante: el agente y el servidor MCP existen como componentes, pero no se ha demostrado una orquestación real de todas las herramientas MCP desde el agente en cada revisión. Por ello, el diagrama describe la arquitectura objetivo y el flujo parcial implementado, no una integración que se haya probado completamente.
4. Servidor MCP: herramientas, recursos y prompts

Herramientas expuestas

Herramienta
Uso
Controles / límites

get_diff
Consultar cambios de Git para
revisar diferencias.
Usa comandos de Git sin shell y con tiempo límite.

read_file
Leer un archivo permitido del
proyecto.
Valida ruta, extensiones permitidas y tamaño de archivo.

search_code
Buscar texto o patrones en el
código.
Restringe la búsqueda al proyecto y evita directorios
excluidos.

get_file_history
Consultar historial de un archivo
mediante Git.
Ejecuta Git con argumentos separados y tiempo límite.

run_tests
Ejecutar pruebas del proyecto.
Usa subprocess sin shell, tiempo límite y entorno
reducido; no es sandbox fuerte.

Recursos

• project://style-guide: guía de estilo que aporta convenciones para la revisión.

• project://risk-catalog: catálogo de riesgos que ayuda a orientar la identificación de problemas.

Prompts

• general_code_review: plantilla de instrucciones para una revisión general del código.

• security_review: plantilla enfocada en riesgos de seguridad.

Las capturas disponibles en evidencias muestran herramientas, recursos y prompts en MCP Inspector. La inspección visual confirma que se encuentran registrados y accesibles desde el inspector; por sí sola no prueba que cada función sea segura ante cualquier entrada.

Límites de entrada y salida

Las rutas se resuelven respecto a la raíz del proyecto y se rechazan rutas que intenten escapar de ella. También se excluyen directorios no deseados y nombres que parezcan contener secretos. La lectura limita extensiones y tamaño. Estas medidas reducen riesgos, pero deben complementarse con pruebas adversariales, permisos mínimos y aislamiento del proceso.

5. Agente y proveedor de modelo

El agente se encuentra en agent/agent.py. La configuración utiliza la variable de entorno GROQ_API_KEY y el nombre de modelo openai/gpt-oss-20b. La clave no debe almacenarse en el repositorio. El archivo .env.example debe contener únicamente un marcador vacío o de ejemplo.

Prueba de conexión

Se confirmó una conexión con Groq y una prueba mínima de solicitud de herramienta. En aquella prueba, el modelo produjo una solicitud para una herramienta llamada consultar_ estado_ proyecto con el argumento proyecto = “Examen final”. Se registraron 145 tokens de entrada y 46 de salida para esa llamada mínima. Estos valores pertenecen a la prueba de conexión y no representan el consumo completo de una revisión ni el coste total del proyecto.

Estado funcional

• Conexión al proveedor y llamada mínima confirmadas.

• El agente básico procesa una solicitud y devuelve una estructura de respuesta.

• No está demostrado que el agente invoque automáticamente las cinco herramientas MCP reales durante la revisión.

• No se ha verificado un ciclo completo de múltiples llamadas a herramientas con manejo exhaustivo de errores.

• No se ha confirmado streaming; por tanto, no se atribuye esa capacidad al sistema.

Fases y límites

El diseño objetivo de una revisión robusta separa Explore (recopilar contexto), Analyze (examinar posibles defectos) y Report (validar y presentar hallazgos). En el estado revisado, estas fases no están implementadas como una orquestación completa e independiente. Los límites de tiempo y número de llamadas deben ser controles efectivos del flujo, no solamente valores anotados en el reporte.

Para producción, el agente debería validar la estructura JSON con un esquema estricto, rechazar categorías no permitidas, limitar el tamaño de salida, manejar respuestas incompletas y diferenciar errores de red, límites del proveedor y respuestas inválidas.

6. Evaluación: conjunto de casos y metodología

El conjunto definido en eval/casos/casos.json contiene 15 casos: 4 de lógica, 3 de seguridad, 2 de rendimiento, 2 de concurrencia y 4 controles limpios. Los controles limpios son importantes para estimar la frecuencia con la que el sistema señala problemas donde no se esperaba un defecto.

Tipo
Cantidad
Objetivo

Lógica
4
Detectar condiciones, límites y resultados incorrectos.

Seguridad
3
Identificar validaciones ausentes o patrones de riesgo.

Rendimiento
2
Detectar operaciones costosas o ineficientes.

Concurrencia
2
Identificar condiciones de carrera u otros riesgos concurrentes.

Controles limpios
4
Comprobar que código sin defecto conocido no genere alertas indebidas.

Total
15
11 casos con defectos y 4 controles limpios.

Métricas

Precisión = TP / (TP + FP). De los hallazgos que el sistema marca como positivos, qué proporción es correcta según la regla de evaluación.

Recall = TP / (TP + FN). De los casos defectuosos conocidos, qué proporción se detecta.

F1 = 2 × (precisión × recall) / (precisión + recall). Resume el equilibrio entre precisión y recall.

Tasa de falsos positivos en controles limpios = controles limpios marcados incorrectamente / total de controles limpios.

En la implementación actual, estas medidas operan a nivel de caso con una regla simplificada: considera detectado un caso cuando aparece una categoría reconocida. No compara de forma exhaustiva cada hallazgo con el defecto exacto de referencia, su ubicación y gravedad. Por ello, las métricas deben interpretarse como resultados preliminares, no como certificación de exactitud semántica.

7. Resultados de evaluación

Archivo de resultado
TP
FP
FN
Precisión
Recal l
F1
Tiempo

eval_run_1_primera.json
10
0
1
1.000
0.909
0.95

2
41.09 s

eval_run_2.json
11
0
0
1.000
1.000
1.00
0
41.96 s

eval_run_3.json
11
0
0
1.000
1.000
1.00
0
34.18 s

eval_post_spec.json
11
0
0
1.000
1.000
1.00
0
31.49 s aprox.

La primera ejecución guardada perdió un caso de lógica, LOG-04. En una ejecución posterior se aclaró la especificación del caso para establecer que una persona de 18 años o más debe ser elegible; el ejemplo de código usa age > 18 y la corrección esperada es age >= 18. Después de esa aclaración, el resultado pasó a 11 TP, 0 FP y 0 FN.

Advertencia de comparabilidad: las primeras ejecuciones y las posteriores a la aclaración de LOG-04 no son estrictamente equivalentes, porque cambió la especificación del caso. No se debe atribuir toda la diferencia a una mejora del modelo sin mantener constante el conjunto, la rúbrica y la configuración.

Variabilidad

En las tres ejecuciones iniciales guardadas, el tiempo total fue aproximadamente 34.18–41.96 segundos. La variación puede depender de latencia de red, carga del proveedor y variación de generación. Los resultados de clasificación fueron estables entre las ejecuciones 2 y 3,pero el tamaño de muestra es pequeño y la regla de detección es simplificada.

El tiempo medio de esas tres ejecuciones es aproximadamente 39.08 segundos. Este promedio describe las ejecuciones guardadas y no garantiza el tiempo de futuras revisiones.

8. Análisis de errores y casos difíciles
Caso LOG-04: límite de edad

El primer resultado registró un falso negativo: el sistema no detectó el defecto de límite en LOG-04. La condición age > 18 excluye a las personas de exactamente 18 años, aunque la especificación exige edad de 18 o más. La corrección es usar age >= 18. Este caso demuestra por qué la especificación y los valores de frontera deben estar explícitos.

Debilidades de la evaluación

• Un caso puede contarse como detectado por la categoría general aunque el modelo no haya señalado el defecto exacto.

• Las métricas agregadas pueden ocultar fallos por tipo de defecto.

• Los resultados no prueban que el sistema identifique correctamente línea, causa, severidad y corrección en todos los casos.

• El conjunto de 15 casos es útil como evaluación inicial, pero pequeño para generalizar a repositorios reales.

• Las corridas que utilizan especificaciones distintas no deben compararse como si fueran idénticas.

Tres riesgos de fallo relevantes

Riesgo
Efecto posible
Mejora recomendada

Defecto correcto,
explicación imprecisa
Alerta difícil de verificar o
corrección equivocada.
Comparar categoría, ubicación, explicación

y evidencia contra ground truth.

Falso negativo en valores límite
Un error real no se reporta.
Añadir pruebas de frontera y casos parametrizados.

Detección por categoría demasiado amplia
Métricas más altas de lo que justificaría una revisión exacta.
Usar matching de hallazgos por tipo, ubicación y condición; revisión manual de desacuerdos.

Los dos últimos riesgos son conclusiones de diseño derivadas de la evaluación disponible, no afirmaciones de que se haya observado cada uno en una ejecución específica.

9. Seguridad: controles implementados y límites

Pruebas existentes

El conjunto de pruebas de seguridad comprueba el rechazo de rutas que salen de la raíz del proyecto, rutas absolutas externas y rutas con byte nulo, además de aceptar una ruta válida dentro del proyecto. Junto con las pruebas del servidor, el resultado local fue de cinco pruebas pasadas.

Amenaza	Control observado	Limitación
Path traversal	Resolución de rutas y rechazo de rutas fuera de la raíz.	Deben mantenerse pruebas para variantes de codificación y enlaces simbólicos.
Lectura de archivos sensibles	Extensiones permitidas, exclusiones y filtro de nombres sospechosos.	El filtro por nombre no detecta todos los secretos posibles.
Comandos del sistema	Subprocess con argumentos separados y sin shell.	El comando todavía ejecuta con los

permisos del usuario del proceso.

Pruebas
potencialmente
peligrosas
Timeout y entorno reducido en run_tests.
No hay evidencia de límites duros de
memoria ni aislamiento de red.

Prompt injection
Prompts de revisión disponibles.
No se ha documentado una prueba
antes/después con resultados
cuantificados.

Prompt injection

Una entrada hostil puede intentar que el modelo ignore las instrucciones de revisión, revele datos o solicite acciones no autorizadas. Una defensa robusta trata el código y los archivos como datos no confiables, mantiene instrucciones de sistema separadas, limita herramientas, valida argumentos y evita que el modelo controle directamente operaciones privilegiadas.

En el estado de evidencia disponible no se certifica un ensayo cuantitativo antes/después de prompt injection. Para completar ese requisito se debe conservar el texto de prueba, resultado sin mitigación, resultado con mitigación y criterio de éxito. No se debe afirmar que la defensa fue eficaz sin esos registros.

Sandbox de run_tests

El timeout y el entorno reducido son medidas útiles, pero no constituyen un sandbox fuerte. Una solución más segura ejecutaría pruebas en un contenedor o entorno aislado con usuario sin privilegios, sistema de archivos limitado, red deshabilitada, límites de CPU/memoria, tiempo máximo y ausencia de secretos. Esas propiedades deben verificarse en el entorno real, no solamente describirse.

10. Integración continua con GitHub Actions

El repositorio público utilizado es: github.com/victormanuelcanasstustc-stack/ai-dev-tools-final. El workflow está en .github/workflows/ci.yml y se activa en push, pull request hacia main/master y ejecución manual (workflow_dispatch).

Pasos del workflow

• Descargar el repositorio con actions/checkout.

• Configurar Python 3.11.

• Actualizar pip e instalar requirements.txt.

• Ejecutar python -m pytest -q.

Incidente de compatibilidad

La primera ejecución remota falló con ModuleNotFoundError para mcp.server.fastmcp. La dependencia estaba sin límite de versión y el runner instaló MCP 2.x, cuya API no era compatible con el servidor existente. Se corrigió la especificación de dependencia para mantener MCP por debajo de la versión 2.
Después, el estudiante informó que las pruebas en GitHub quedaron correctas.

El workflow automatiza las pruebas existentes; no ejecuta una revisión completa con Groq ni publica comentarios de revisión en pull requests. Tampoco se dispone aquí de un identificador de ejecución o captura de la ejecución verde para certificar visualmente dos corridas reales. Para la entrega, deben adjuntarse capturas de dos ejecuciones exitosas distintas desde la pestaña Actions.
Reproducción local

Desde la raíz del proyecto, con el entorno virtual activo, ejecutar:
`python -m pytest -q`
. En GitHub Actions se ejecuta el mismo comando en un runner Ubuntu con Python 3.11.

11. Rendimiento, latencia y coste

Las métricas de tiempo de evaluación registradas fueron 41.09 s, 41.96 s y 34.18 s para tres archivos de ejecución inicial. La evaluación posterior a la especificación de LOG-04 se completó en aproximadamente 31.49 s. Estos tiempos son de la ejecución completa del evaluador, no de una llamada individual del modelo.

Métrica	Dato disponible	Interpretación
Tiempo por corrida	31.49–41.96 s según archivo	Tiempo total de evaluación de 15 casos; varía entre corridas.
Tokens de prueba mínima	145 entrada / 46 salida	Corresponde a una llamada de prueba de herramienta, no a toda la evaluación.

Tokens totales de
evaluación
No consolidados en
evidencia disponible
No se debe estimar como si fuera consumo medido.

Coste monetario
No calculado con
factura/registro de uso
No se afirma un coste total. Debe consultarse el panel del proveedor.

Streaming
No verificado
No se atribuye soporte streaming a la implementación actual.

Como el modelo se consumió mediante un proveedor externo, el tiempo total incluye componentes de red y servicio remoto. Para un informe de costes completo habría que registrar tokens de cada llamada, número de llamadas, modelo y tarifa aplicable en la fecha de uso. La prueba mínima de conexión no es suficiente para inferir el coste de la evaluación.

12. Decisiones técnicas, errores y aprendizaje

Decisiones

• Adoptar la Variante C autorizada por la falta de acceso a Claude API.

• Utilizar Groq y configurar openai/gpt-oss-20b.

• Mantener el servidor MCP en Python y conservar herramientas, recursos y prompts separados.

• Limitar la versión de MCP para que el runner remoto use la API compatible con el servidor.

• Conservar resultados JSON y capturas como evidencia reproducible.

• Usar GitHub Actions para automatizar las pruebas existentes.

Errores y resolución

Problema
Causa
Resolución / aprendizaje

GitHub Actions falló al
importar FastMCP

MCP 2.x instalado por
dependencia sin límite.

Fijar la dependencia a la línea compatible
MCP 1.x y repetir la ejecución.

LOG-04 no detectado inicialmente
Condición de frontera y
especificación que se aclaró después.
Aclarar edad mínima inclusiva y usar age >=
18 como condición correcta.

Advertencias LF/CRLF en
Windows
Diferencias normales de finales de
línea entre Windows y runner
Linux.
No son errores de ejecución; revisar el diff si
aparece ruido innecesario.

Comandos de inspección
con errores de sintaxis
Comando de PowerShell mal
copiado.
Usar bloques sencillos, revisar el resultado
antes de continuar.

No se incluyen hashes de commits específicos porque no se han recopilado en la evidencia disponible para este documento. Se pueden añadir mediante `git log --oneline` si la rúbrica exige trazabilidad por commit.

13. Instrucciones de instalación y ejecución

Requisitos

• Python 3.11 recomendado para reproducir CI.

• Git.

• Cuenta/proyecto en Groq si se quiere ejecutar el agente con el proveedor remoto.

• Clave de API guardada como variable de entorno; nunca en el repositorio.

Instalación local en Windows PowerShell

Desde la raíz del repositorio:

```
python
-m
venv .venv

.\.venv\Scripts\Activate.ps1

python
-m
pip
install
--upgrade
pip

pip
install
-r requirements.txt

python
-m
pytest
-q
```

Variables de entorno

El archivo .env.example documenta GROQ_API_KEY y GROQ_MODEL. Para ejecutar el agente, se debe crear un archivo .env local o definir la variable en el entorno. El archivo real .env debe permanecer ignorado por Git. No se debe copiar la clave a README, capturas públicas, logs ni al informe.

Ejecutar evaluación

Ejecutar
`python eval/run_eval.py`
desde la raíz, después de revisar el script y confirmar que la clave de Groq está disponible. Guardar cada ejecución con un nombre distinto; evitar sobrescribir resultados anteriores.

El evaluador actual puede escribir un archivo de resultado fijo, por lo que para reproducibilidad conviene cambiar el nombre de salida o copiar cada resultado inmediatamente después de la ejecución.

15. Limitaciones, trabajo futuro y conclusiones

Limitaciones conocidas

- El agente no está demostrado como un orquestador completo de las cinco herramientas MCP.

- Las fases Explore, Analyze y Report no se han verificado como fases independientes y completas.

- El evaluador usa una regla simplificada de detección por caso/categoría; no compara cada hallazgo de forma exacta.

- No hay evidencia suficiente de prueba de prompt injection antes/después con resultados cuantificados.

- run_tests tiene controles básicos, pero no aislamiento fuerte de memoria/red/permisos.

- GitHub Actions ejecuta pytest; no se ha implementado un sistema completo de comentarios de revisión sobre pull requests.

- No se han consolidado los tokens y costes monetarios de todas las corridas.

- Las dos ejecuciones verdes deben documentarse con capturas verificables.

Mejoras prioritarias

- Integrar las herramientas MCP en un ciclo real de agente con límites efectivos de tiempo y llamadas.

- Validar el JSON con un esquema estricto y normalizar categorías, ubicación, severidad y evidencia.

- Mejorar el evaluador con matching por tipo de defecto y ubicación, y reportar métricas por categoría.

- Añadir pruebas de prompt injection antes/después y registrar las respuestas.

- Ejecutar run_tests en contenedor aislado sin red y con límites de CPU/memoria.

- Registrar tokens, latencia por llamada, total de llamadas y coste según la tarifa vigente.

- Conservar dos capturas de CI exitoso y los hashes de commits relacionados.

Conclusiones

El proyecto demuestra una base funcional de servidor MCP, controles básicos de rutas, conexión a un modelo alternativo, evaluación con casos conocidos y automatización de pruebas. La corrección de la dependencia de MCP permitió resolver una incompatibilidad entre el entorno local y el runner remoto.

La principal lección es que las métricas de un agente deben leerse junto con el método de evaluación y las limitaciones del sistema. Un resultado perfecto en un conjunto pequeño no demuestra que una herramienta detecte todos los defectos reales. La seguridad también requiere más que filtrar rutas y establecer un timeout: se necesita aislamiento, validación, permisos mínimos y pruebas adversariales.

La solución es una base académica que puede ampliarse con integración completa de herramientas, evaluación más estricta, mejores controles de seguridad y observabilidad de costes y rendimiento.

16. Diez preguntas conceptuales

1. ¿Qué es una ventana de contexto?

Es la cantidad de información que un modelo puede considerar en una interacción. Incluye instrucciones, mensajes y contenido añadido al contexto. Como es limitada, conviene seleccionar la información relevante y evitar enviar datos innecesarios.

2. ¿Qué son los subagentes?

Son agentes especializados que se encargan de una parte de una tarea y devuelven resultados al flujo principal. Funcionan mejor cuando tienen responsabilidades definidas y sus resultados se verifican.

3. ¿Para qué sirve CLAUDE.md?

Es un archivo de contexto persistente utilizado por Claude Code para describir el proyecto, las convenciones, los comandos y las restricciones de trabajo. En esta variante se utilizó otro proveedor, por lo que no se afirma que CLAUDE.md controle el agente Groq.

4. ¿Cuál es la diferencia entre system prompt y user prompt?

El system prompt establece reglas y comportamiento general del asistente. El user prompt expresa la solicitud concreta. Mantenerlos separados ayuda a diferenciar las reglas de la tarea y los datos que aporta la persona.

5. ¿Cuál es el ciclo de una herramienta en una API?

La aplicación envía contexto al modelo; el modelo solicita una herramienta con argumentos estructurados; la aplicación valida y ejecuta la herramienta; devuelve el resultado al modelo; y el modelo produce una respuesta final o solicita otra herramienta.

6. ¿Qué es MCP?
El Model Context Protocol es un protocolo para conectar aplicaciones de IA con servidores que ofrecen herramientas, recursos y prompts mediante una interfaz común.

7. ¿Qué diferencia existe entre tools, resources y prompts en MCP?

Una tool permite realizar una acción; un resource expone información consultable; y un prompt ofrece una plantilla de instrucciones reutilizable. En este proyecto se usan para separar operaciones, contexto y orientación de revisión.

8. ¿Qué es prompt injection?

Es un intento de introducir instrucciones maliciosas en contenido que el modelo procesa para cambiar su comportamiento previsto. Se mitiga separando instrucciones confiables de datos, limitando herramientas, validando entradas y exigiendo autorización para acciones sensibles.

9. ¿Cómo se realiza una evaluación objetiva de un LLM?

Se define un conjunto de casos y una referencia esperada antes de ejecutar el modelo, se aplican criterios verificables y se registran errores y métricas. Es importante mantener constantes los casos entre corridas y revisar manualmente los desacuerdos.

10. ¿Cuáles son limitaciones importantes de los LLM?

Pueden equivocarse, inventar información, interpretar mal el contexto y producir respuestas convincentes pero incorrectas. Su salida debe validarse con esquemas, pruebas, herramientas controladas y revisión humana.